

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

KHUSHI SHRIVASTAV(1BF24CS139)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Khushi Shrivastav(**1BF24CS139**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-7
2	Implement Johnson Trotter algorithm to generate permutations.	8-10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	11-15
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	16-18
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19-21
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	22-24
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	25-28
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29-31 32-34
9	Implement Fractional Knapsack using Greedy technique.	35-37
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	38-40
11	Implement "N-Queens Problem" using Backtracking.	41-42

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 10

int main() {
    int n, i, j;
    int indegree[MAX] = {0};
    int graph[MAX][MAX];
    int order[MAX], count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if(graph[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
    while(count < n) {
        int found = 0;
        for(i = 0; i < n; i++) {
            if(indegree[i] == 0) {
                order[count++] = i;
                indegree[i] = -1;
                for(j = 0; j < n; j++) {
                    if(graph[i][j] == 1) {
                        indegree[j]--;
                    }
                }
            }
        }
    }
}
```

```

        }
    }
    found = 1;
}
}
if(!found) break;
}
if(count < n) {
    printf("No Topological Sequence (cycle detected)\n");
} else {
    printf("Topological Order: ");
    for(i = 0; i < n; i++) {
        printf("%d ", order[i]);
    }
    printf("\n");
}
return 0;
}

```

Screenshot of the output:

```

"C:\Users\Khushi Shrivastav\C"
Enter number of vertices: 6
Enter adjacency matrix:
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Topological Order: 0 1 2 3 4 5

Process returned 0 (0x0)   execution time : 216.452 s
Press any key to continue.

```

LEETCODE 207:

207. Course Schedule Solved

Medium Topics Companies Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`

18.1K 350 74 Online

Accepted 54 / 54 testcases passed
KhushiShrivastav submitted at Jun 02, 2026 17:05

Runtime 39 ms | Beats 30.55%
Memory 126.14 MB | Beats 6.03%

Testcase Test Result
Accepted Runtime: 0 ms

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {
```

```
    int *indegree = (int *)calloc(numCourses, sizeof(int));
```

```
    int **adj = (int **)malloc(numCourses * sizeof(int *));
```

```
    int *adjSize = (int *)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        adj[i] = (int *)malloc(prerequisitesSize * sizeof(int));
```

```
    }
```

```
    for (int i = 0; i < prerequisitesSize; i++) {
```

```
        int course = prerequisites[i][0];
```

```
        int prereq = prerequisites[i][1];
```

```
        adj[prereq][adjSize[prereq]++] = course;
```

```
        indegree[course]++;
```

```
    }
```

```
    int *queue = (int *)malloc(numCourses * sizeof(int));
```

```
    int front = 0, rear = 0;
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        if (indegree[i] == 0)
```

```

        queue[rear++] = i;
    }
    int count = 0;
    while (front < rear) {
        int node = queue[front++];
        count++;
        for (int i = 0; i < adjSize[node]; i++) {
            int next = adj[node][i];
            indegree[next]--;
            if (indegree[next] == 0)
                queue[rear++] = next;
        }
    }
    for (int i = 0; i < numCourses; i++)
        free(adj[i]);
    free(adj);
    free(adjSize);
    free(indegree);
    free(queue);
    return count == numCourses;
}

```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

#define LEFT 0
#define RIGHT 1

void print(int a[], int dir[], int n) {
    for(int i = 0; i < n; i++) {
        if(dir[a[i]] == LEFT)
            printf("%d<- ", a[i]);
        else
            printf("%d-> ", a[i]);
    }
    printf("\n");
}

int getMobile(int a[], int dir[], int n) {
    int mobile = 0;
    for(int i = 0; i < n; i++) {
        if(dir[a[i]] == LEFT && i > 0) {
            if(a[i] > a[i-1] && a[i] > mobile)
                mobile = a[i];
        }
        if(dir[a[i]] == RIGHT && i < n-1) {
            if(a[i] > a[i+1] && a[i] > mobile)
                mobile = a[i];
        }
    }
    return mobile;
}

int findPos(int a[], int n, int mobile) {
```



```

    for(int i = 0; i < n; i++) {
        if(a[i] == mobile)
            return i;
    }
    return -1;
}

void johnsonTrotter(int n) {
    int a[n], dir[n+1];
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i+1] = LEFT;
    }
    print(a, dir, n);
    while(1) {
        int mobile = getMobile(a, dir, n);
        if(mobile == 0)
            break;
        int pos = findPos(a, n, mobile);
        if(dir[mobile] == LEFT) {
            int temp = a[pos];
            a[pos] = a[pos-1];
            a[pos-1] = temp;
            pos = pos - 1;
        } else {
            int temp = a[pos];
            a[pos] = a[pos+1];
            a[pos+1] = temp;
            pos = pos + 1;
        }
    }
}

```

```

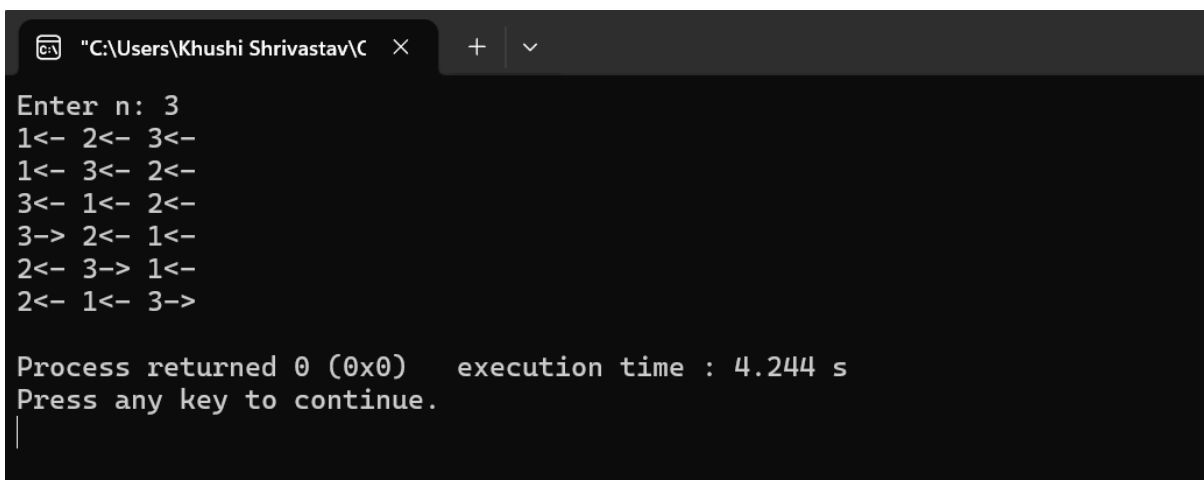
        for(int i = 1; i <= n; i++) {
            if(i > mobile) {
                dir[i] = !dir[i];
            }
        }
        print(a, dir, n);
    }
}

int main() {
    int n;

    printf("Enter n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}

```

Screenshot of the output:



```

Enter n: 3
1<- 2<- 3<-
1<- 3<- 2<-
3<- 1<- 2<-
3-> 2<- 1<-
2<- 3-> 1<-
2<- 1<- 3->

Process returned 0 (0x0)   execution time : 4.244 s
Press any key to continue.
|

```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int arr[], int lb, int mid, int ub) {

    int n1 = mid - lb + 1;

    int n2 = ub - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)

        L[i] = arr[lb + i];

    for (int j = 0; j < n2; j++)

        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = lb;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j])

            arr[k++] = L[i++];

        else

            arr[k++] = R[j++];

    }

    while (i < n1)

        arr[k++] = L[i++];

    while (j < n2)

        arr[k++] = R[j++];

}

void mergeSort(int arr[], int lb, int ub) {

    if (lb < ub) {

        int mid = (lb + ub) / 2;
```

```

        mergeSort(arr, lb, mid);
        mergeSort(arr, mid + 1, ub);
        merge(arr, lb, mid, ub);
    }
}

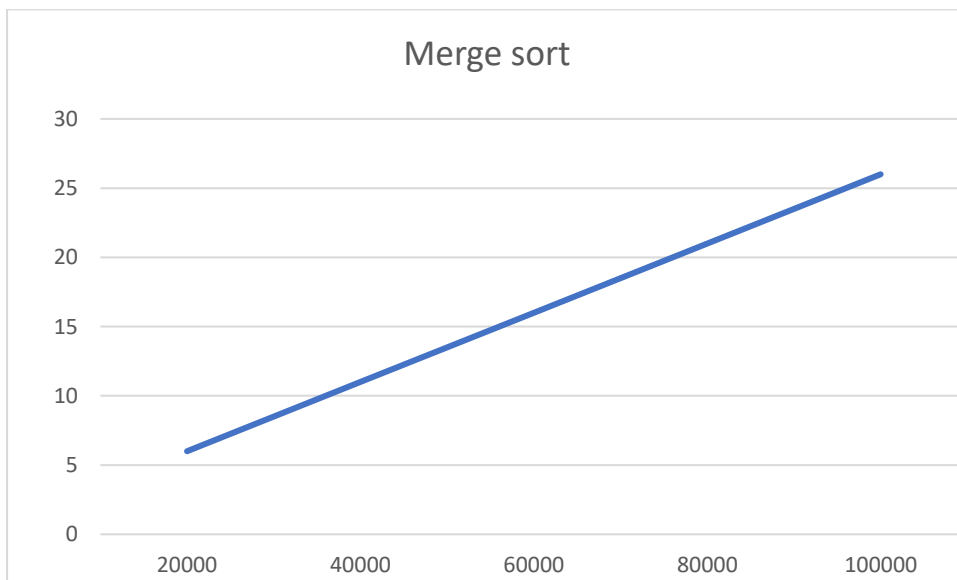
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    clock_t start = clock();
    mergeSort(arr, 0, n - 1);
    clock_t end = clock();
    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("Time taken: %.6f seconds\n", time_taken);
    return 0;
}

```

Screenshot of the output:

```
"C:\Users\Khushi Shrivastav\C  ×  +  v
Enter number of elements: 20000
Time taken to sort 20000 elements = 0.006000 seconds

Process returned 0 (0x0)   execution time : 4.735 s
Press any key to continue.
|
```



LEETCODE 912:

912. Sort an Array Solved

Medium Topics Companies

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in functions** in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`
Output: `[1,2,3,5]`
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: `nums = [5,1,1,2,0,0]`
Output: `[0,0,1,1,2,5]`
Explanation: Note that the values of `nums` are not necessarily unique.

Accepted 21 / 21 testcases passed
KhushiShrivastav submitted at May 23, 2026 23:35

Runtime: 56 ms | Beats 62.77%
Memory: 59.18 MB | Beats 91.77%

Testcase Test Result

You must run your code first

```
void merge(int nums[], int left, int mid, int right){
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    for(int i = 0; i < n1; i++)
```

```
        L[i] = nums[left + i];
```

```
    for(int j = 0; j < n2; j++)
```

```
        R[j] = nums[mid + 1 + j];
```

```
    int i = 0, j = 0, k = left;
```

```
    while(i < n1 && j < n2)
```

```
    {
```

```
        if(L[i] <= R[j])
```

```
            nums[k++] = L[i++];
```

```
        else
```

```
            nums[k++] = R[j++];
```

```
    }
```

```
    while(i < n1)
```

```
        nums[k++] = L[i++];
```

```
    while(j < n2)
```

```
        nums[k++] = R[j++];
    }
void mergeSort(int nums[], int left, int right)
{
    if(left < right)
    {
        int mid = left + (right - left) / 2;
        mergeSort(nums, left, mid);
        mergeSort(nums, mid + 1, right);
        merge(nums, left, mid, right);
    }
}
int* sortArray(int* nums, int numsSize, int* returnSize)
{
    mergeSort(nums, 0, numsSize - 1);
    *returnSize = numsSize;
    return nums;
}
```

Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choosing last element as pivot
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int N;
    printf("Enter number of elements: ");
    scanf("%d", &N);
```



```

int arr[N];

printf("Enter the elements:\n");
for (int i = 0; i < N; i++) {
    scanf("%d", &arr[i]);
}

clock_t start, end;
start = clock();
quickSort(arr, 0, N - 1);
end = clock();

printf("Sorted array:\n");
for (int i = 0; i < N; i++) {
    printf("%d ", arr[i]);
}

double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nTime taken: %f seconds\n", time_taken);

return 0;
}

```

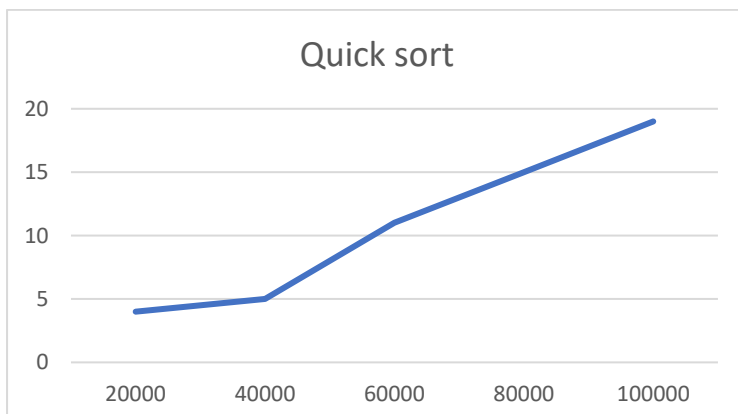
Screenshot of the output:

```

C:\Users\Khushi Shrivastav\C > Enter number of elements: 20000
Time taken to sort 20000 elements = 0.005000 seconds

Process returned 0 (0x0)   execution time : 3.266 s
Press any key to continue.
|

```



LEETCODE 455:

455. Assign Cookies Solved

Easy Topics Companies

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input: $g = [1,2,3]$, $s = [1,1]$
Output: 1
Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to output 1.

Accepted 25 / 25 testcases passed
KhushiShrivastav submitted at Jun 02, 2026 17:23

Runtime: 27 ms Beats 22.88%
Memory: 12.52 MB Beats 74.52%

Code: C

```
1 #include <stdlib.h>
```

```
#include <stdlib.h>
```

```
int compare(const void *a, const void *b){  
    return (*(int *)a - *(int *)b);}
```

```
int findContentChildren(int* g, int gSize, int* s, int sSize){  
    qsort(g, gSize, sizeof(int), compare);  
    qsort(s, sSize, sizeof(int), compare);  
    int i = 0; // child  
    int j = 0; // cookie  
    int count = 0;  
    while (i < gSize && j < sSize){  
        if (s[j] >= g[i]){  
            count++;  
            i++;  
            j++; }  
        else{  
            j++;}  
    }  
    return count;  
}
```

Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void siftUp(int a[], int i) {
    while(i > 0) {
        int parent = (i - 1) / 2;
        if(a[parent] < a[i]) {
            swap(&a[parent], &a[i]);
            i = parent;
        } else {
            break;
        }
    }
}

void buildHeap(int a[], int n) {
    for(int i = 1; i < n; i++) {
        siftUp(a, i);
    }
}

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if(left < n && a[left] > a[largest])
```

```

        largest = left;
    if(right < n && a[right] > a[largest])
        largest = right;
    if(largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    buildHeap(a, n);
    for(int i = n-1; i > 0; i--) {
        swap(&a[0], &a[i]);
        heapify(a, i, 0);
    }
}

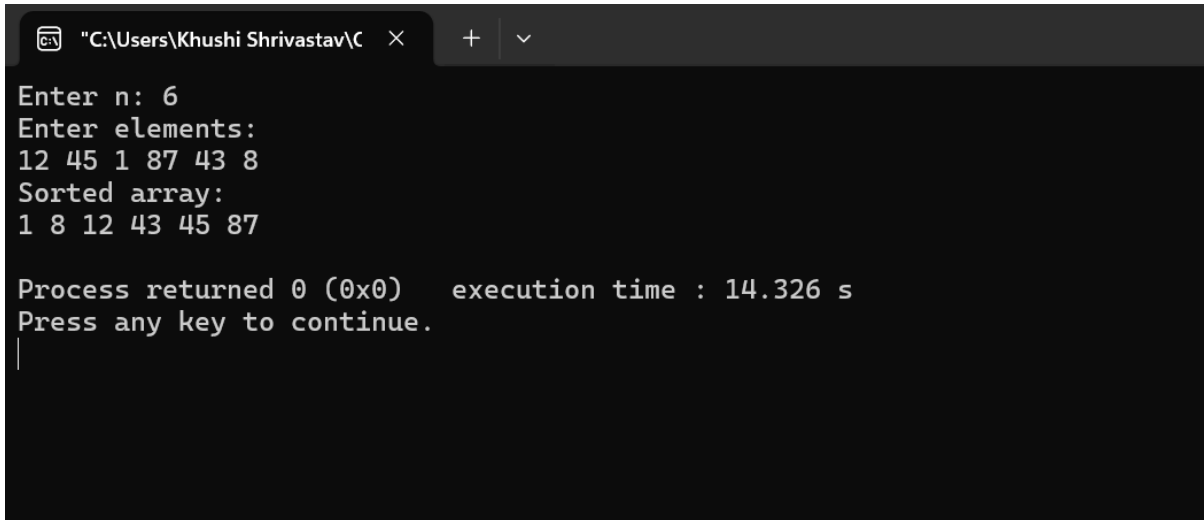
void print(int a[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    heapSort(a, n);
}

```

```
printf("Sorted array:\n");  
print(a, n);  
return 0;  
}
```

Screenshot of the output:



```
"C:\Users\Khushi Shrivastav\C" × + v  
Enter n: 6  
Enter elements:  
12 45 1 87 43 8  
Sorted array:  
1 8 12 43 45 87  
  
Process returned 0 (0x0) execution time : 14.326 s  
Press any key to continue.  
|
```

Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            else if (wt[i-1] <= w) {
                int include = val[i-1] + dp[i-1][w - wt[i-1]];
                int exclude = dp[i-1][w];
                dp[i][w] = (include > exclude) ? include : exclude;
            }
            else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }
    int selected[n];
    for (int i = 0; i < n; i++)
        selected[i] = 0;
    int i = n;
    int w = W;
    while (i > 0 && w > 0) {
        if (dp[i][w] != dp[i-1][w]) {
            selected[i-1] = 1;
            w = w - wt[i-1];
        }
    }
}
```

```

    }
    i--;
}
printf("\nItems included in Knapsack:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", selected[i]);
}
printf("\n");
return dp[n][W];
}
int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    printf("Enter profit of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    int result = knapsack(W, wt, val, n);
    printf("\nMaximum value in Knapsack = %d\n", result);
    return 0;
}

```

Screenshot of the output:

```
"C:\Users\Khushi Shrivastav\C x + v
Enter number of items: 4
Enter profit of items:
1 4 5 7
Enter weights of items:
1 3 4 5
Enter knapsack capacity: 7

Items included in Knapsack:
0 1 1 0

Maximum value in Knapsack = 9

Process returned 0 (0x0) execution time : 52.429 s
Press any key to continue.
```

LEETCODE 70:

The screenshot shows the LeetCode 70: Climbing Stairs problem page. The problem description states: "You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?" Examples provided are: Example 1: Input: $n = 2$, Output: 2; Explanation: There are two ways to climb to the top. 1. 1 step + 1 step, 2. 2 steps. Example 2: Input: $n = 3$, Output: 3; Explanation: There are three ways to climb to the top. 1. 1 step + 1 step + 1 step, 2. 1 step + 2 steps, 3. 2 steps + 1 step. The submission result shows "Accepted" with 45/45 testcases passed, runtime of 0ms (Beats 100.00%), and memory of 8.56 MB (Beats 36.81%). The code is in C.

```
int climbStairs(int n){
    if(n == 1)
        return 1;
    int first = 1;
    int second = 2;
    for(int i = 3; i <= n; i++)
    {
        int third = first + second;
        first = second;
        second = third;
    }
    return second;
}
```


Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define INF 99999
#define MAX 100

void printSolution(int dist[MAX][MAX], int V) {
    printf("\nShortest distance matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

void floydWarshall(int graph[MAX][MAX], int V) {
    int dist[MAX][MAX];

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```

        }
    }
}

for (int i = 0; i < V; i++) {
    if (dist[i][i] < 0) {
        printf("\nNegative cycle detected!\n");
        return;
    }
}

printSolution(dist, V);
}

int main() {
    int V;
    int graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("\nEnter adjacency matrix (-1 for no edge):\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == -1)
                graph[i][j] = INF;
        }
    }

    for (int i = 0; i < V; i++)
        graph[i][i] = 0;

    floydWarshall(graph, V);
    return 0;}

```

Screenshot of the output:

```
"C:\Users\Khushi Shrivastav\C  ×  +  ∨  
Enter number of vertices: 4  
Enter adjacency matrix (-1 for no edge):  
0 5 -1 10  
-1 0 3 -1  
-1 -1 3 1  
-1 -1 -1 0  
  
Shortest distance matrix:  
    0      5      8      9  
INF      0      3      4  
INF     INF      0      1  
INF     INF     INF      0  
  
Process returned 0 (0x0)   execution time : 85.731 s  
Press any key to continue.  
|
```

LEETCODE 821:

821. Shortest Distance to a Character Solved

Given a string *s* and a character *c* that occurs in *s*, return an array of integers *answer* where *answer[i]* is the distance from index *i* to the closest occurrence of character *c* in *s*.

The distance between two indices *i* and *j* is $\text{abs}(i - j)$, where *abs* is the absolute value function.

Example 1:

Input: *s* = "loveleetcode", *c* = "e"
Output: [3,2,1,0,1,0,1,2,1,0]
Explanation: The character 'e' appears at indices 3, 5, 6, and 11 (0-indexed).
The closest occurrence of 'e' for index 0 is at index 3, so the distance is $\text{abs}(0 - 3) = 3$.
The closest occurrence of 'e' for index 1 is at index 3, so the distance is $\text{abs}(1 - 3) = 2$.
For index 4, there is a tie between the 'e' at index 3 and the 'e' at index 5, but the distance is still the same: $\text{abs}(4 - 3) = \text{abs}(4 - 5) = 1$.

Accepted 76 / 76 testcases passed
KhushiShrivastav submitted at May 24, 2026 20:33

Runtime: 0 ms Beats 100.00%
Memory: 11.62 MB Beats 53.40%

Code: C

```
1 int* shortestToChar(char* s, char c, int* returnSize)
```

```
int* shortestToChar(char* s, char c, int* returnSize){
    int n = strlen(s);
    int *answer = (int*)malloc(n * sizeof(int));
    int prev = -10000;
    for(int i = 0; i < n; i++){
        if(s[i] == c)
            prev = i;
        answer[i] = i - prev;
    }
    prev = 10000;
    for(int i = n - 1; i >= 0; i--){
        if(s[i] == c)
            prev = i;
        if(prev - i < answer[i])
            answer[i] = prev - i;
    }
    *returnSize = n;
    return answer;
}
```

Lab program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

#include <limits.h>

int minKey(int key[], int mstSet[], int V) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}

void primMST(int graph[50][50], int V) {
    int parent[V], key[V], mstSet[V];
    int totalCost = 0;

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
    }
}
```

```

    }
}

printf("\nEdges in MST:\n");
for (int i = 1; i < V; i++) {
    printf("%d - %d : %d\n", parent[i], i, graph[i][parent[i]]);
    totalCost += graph[i][parent[i]];
}
printf("Total cost = %d\n", totalCost);
}

int main() {
    int V;
    int graph[50][50];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix (enter 0 if no edge, otherwise enter cost):\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    primMST(graph, V);

    return 0;
}

```

Screenshot of the output:

```
"C:\Users\Khushi Shrivastav\C  ×  +  v
Enter number of vertices: 5
Enter adjacency matrix (enter 0 if no edge, otherwise enter cost):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in MST:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
1 - 4 : 5
Total cost = 16

Process returned 0 (0x0)   execution time : 44.524 s
Press any key to continue.
|
```

Lab program 8b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

#include <stdlib.h>

struct Edge {
    int src,dest,weight;
};

struct Subset {
    int parent;
};

int find(struct Subset subsets[],int i){
    while(subsets[i].parent!=i)
        i=subsets[i].parent;
    return i;
}

void Union(struct Subset subsets[],int x,int y){
    int xroot=find(subsets,x);
    int yroot=find(subsets,y);
    subsets[xroot].parent=yroot;
}

int compare(const void *a,const void *b){
    return ((struct Edge*)a)->weight-((struct Edge*)b)->weight;
}

void KruskalMST(struct Edge edges[],int V,int E){
    struct Edge result[V];
    struct Subset subsets[V];
    qsort(edges,E,sizeof(struct Edge),compare);
    for(int i=0;i<V;i++)
        subsets[i].parent=i;
```



```

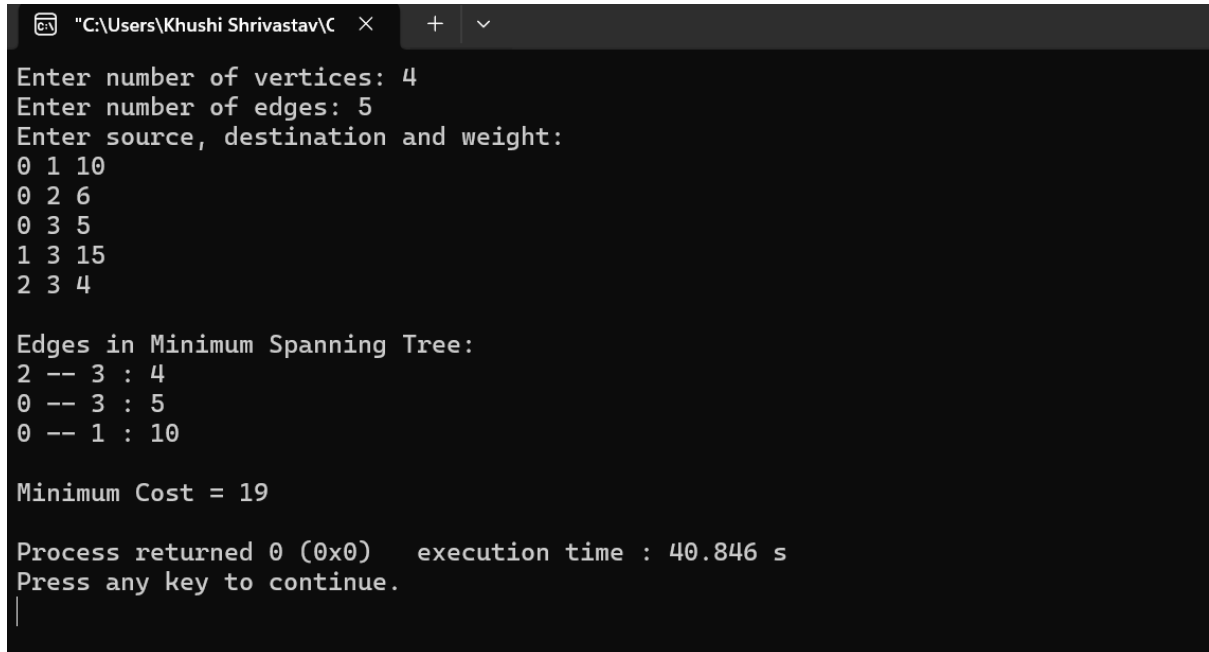
int e=0;
int i=0;
while(e<V-1&& i<E){
    struct Edge next=edges[i++];
    int x=find(subsets,next.src);
    int y=find(subsets,next.dest);
    if(x!=y){
        result[e++]=next;
        Union(subsets,x,y);
    }
}
printf("\nEdges in MST:\n");
int total=0;
for(i=0;i<e;i++){
    printf("%d -- %d == %d\n",result[i].src,result[i].dest,result[i].weight);
    total+=result[i].weight;
}
printf("Total Cost = %d\n",total);
}

int main(){
    int V;
    printf("Enter number of vertices: ");
    scanf("%d",&V);
    int E=V*(V-1)/2;
    struct Edge edges[E];
    printf("Enter edges (src dest weight):\n");
    for(int i=0;i<E;i++){
        scanf("%d %d %d",&edges[i].src,&edges[i].dest,&edges[i].weight);
    }
}

```

```
KruskalMST(edges,V,E);  
  
return 0;  
  
}
```

Screenshot of the output:



```
"C:\Users\Khushi Shrivastav\C  ×  +  v  
Enter number of vertices: 4  
Enter number of edges: 5  
Enter source, destination and weight:  
0 1 10  
0 2 6  
0 3 5  
1 3 15  
2 3 4  
  
Edges in Minimum Spanning Tree:  
2 -- 3 : 4  
0 -- 3 : 5  
0 -- 1 : 10  
  
Minimum Cost = 19  
  
Process returned 0 (0x0)   execution time : 40.846 s  
Press any key to continue.  
|
```

Lab program 9:

Implement Fractional Knapsack using Greedy technique

```
#include <stdio.h>

struct Item{int weight;int value;};

int main(){
    int n,capacity;

    printf("Enter number of items: ");
    scanf("%d",&n);

    struct Item items[n];

    for(int i=0;i<n;i++){
        printf("Enter weight and value of item %d: ",i+1);
        scanf("%d %d",&items[i].weight,&items[i].value);
    }

    printf("Enter knapsack capacity: ");
    scanf("%d",&capacity);

    double ratio[n];

    for(int i=0;i<n;i++)ratio[i]=(double)items[i].value/items[i].weight;

    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(ratio[i]<ratio[j]){
                double tempR=ratio[i];ratio[i]=ratio[j];ratio[j]=tempR;
                struct Item temp=items[i];items[i]=items[j];items[j]=temp;
            }
        }
    }

    double maxVal=0.0;int remaining=capacity;

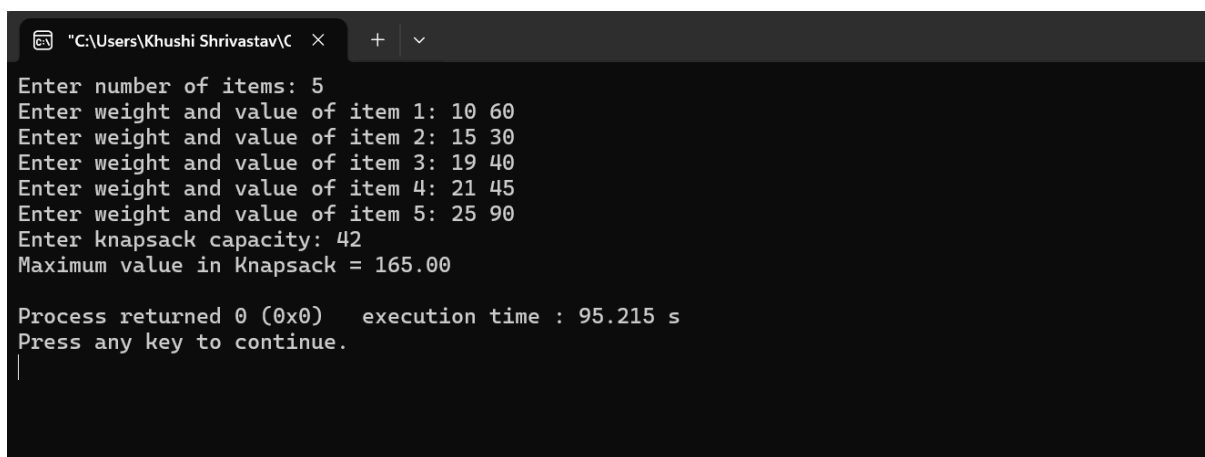
    for(int i=0;i<n;i++){
        if(items[i].weight<=remaining){
            remaining-=items[i].weight;
```

```
        maxValue+=items[i].value;
    }else{
        maxValue+=ratio[i]*remaining;
        break;
    }
}

printf("Maximum value in Knapsack = %.2f\n",maxValue);

return 0;
}
```

Screenshot of the output:

A screenshot of a Windows command prompt window with a dark background. The window title bar shows the file path "C:\Users\Khushi Shrivastav\C...". The terminal displays the following text: "Enter number of items: 5", "Enter weight and value of item 1: 10 60", "Enter weight and value of item 2: 15 30", "Enter weight and value of item 3: 19 40", "Enter weight and value of item 4: 21 45", "Enter weight and value of item 5: 25 90", "Enter knapsack capacity: 42", "Maximum value in Knapsack = 165.00", "Process returned 0 (0x0) execution time : 95.215 s", and "Press any key to continue." followed by a vertical bar cursor.

```
"C:\Users\Khushi Shrivastav\C...
Enter number of items: 5
Enter weight and value of item 1: 10 60
Enter weight and value of item 2: 15 30
Enter weight and value of item 3: 19 40
Enter weight and value of item 4: 21 45
Enter weight and value of item 5: 25 90
Enter knapsack capacity: 42
Maximum value in Knapsack = 165.00

Process returned 0 (0x0) execution time : 95.215 s
Press any key to continue.
|
```

LEETCODE 1710:

1710. Maximum Units on a Truck Solved

You are assigned to put some amount of boxes onto **one** truck. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxesi, numberUnitsPerBoxi]`.

- `numberOfBoxesi` is the number of boxes of type `i`.
- `numberUnitsPerBoxi` is the number of units in each box of the type `i`.

You are also given an integer `truckSize`, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return the **maximum** total number of **units** that can be put on the truck.

Example 1:

Input: `boxTypes = [[1,3],[2,2],[3,1]], truckSize = 4`
Output: 8
Explanation: There are:
- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.

Submission Result: Accepted 77 / 77 testcases passed. Runtime: 1 ms, Beats: 72.60%. Memory: 11.92 MB, Beats: 34.93%.

```
int compare(const void *a, const void *b){

    int *x = *(int **)a;

    int *y = *(int **)b;

    return y[1] - x[1];}

int maximumUnits(int** boxTypes, int boxTypesSize,

                 int* boxTypesColSize, int truckSize){

    qsort(boxTypes, boxTypesSize, sizeof(int *), compare);

    int totalUnits = 0;

    for(int i = 0; i < boxTypesSize; i++){

        int boxes = boxTypes[i][0];

        int units = boxTypes[i][1];

        if(truckSize >= boxes){

            totalUnits += boxes * units;

            truckSize -= boxes;}

        else{

            totalUnits += truckSize * units;

            break;

        }

    }

    return totalUnits;

}
```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#include <limits.h>

#define V 10 // maximum number of vertices

int minDistance(int dist[], int sptSet[], int vertices) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < vertices; v++) {
        if (sptSet[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

void printSolution(int dist[], int vertices, int src) {
    printf("Vertex \t Distance from Source %d\n", src);
    for (int i = 0; i < vertices; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}

void dijkstra(int graph[V][V], int src, int vertices) {
    int dist[vertices]; // shortest distance from src
    int sptSet[vertices]; // whether vertex is included in shortest path tree
    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
        sptSet[i] = 0;
    }
    dist[src] = 0;
```

```

for (int count = 0; count < vertices - 1; count++) {
    int u = minDistance(dist, sptSet, vertices);
    sptSet[u] = 1;
    for (int v = 0; v < vertices; v++) {
        if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
            && dist[u] + graph[u][v] < dist[v]) {
            dist[v] = dist[u] + graph[u][v];
        }
    }
}

printSolution(dist, vertices, src);
}

int main() {
    int vertices;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices)

    int graph[V][V];
    printf("Enter adjacency matrix (0 if no edge):\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

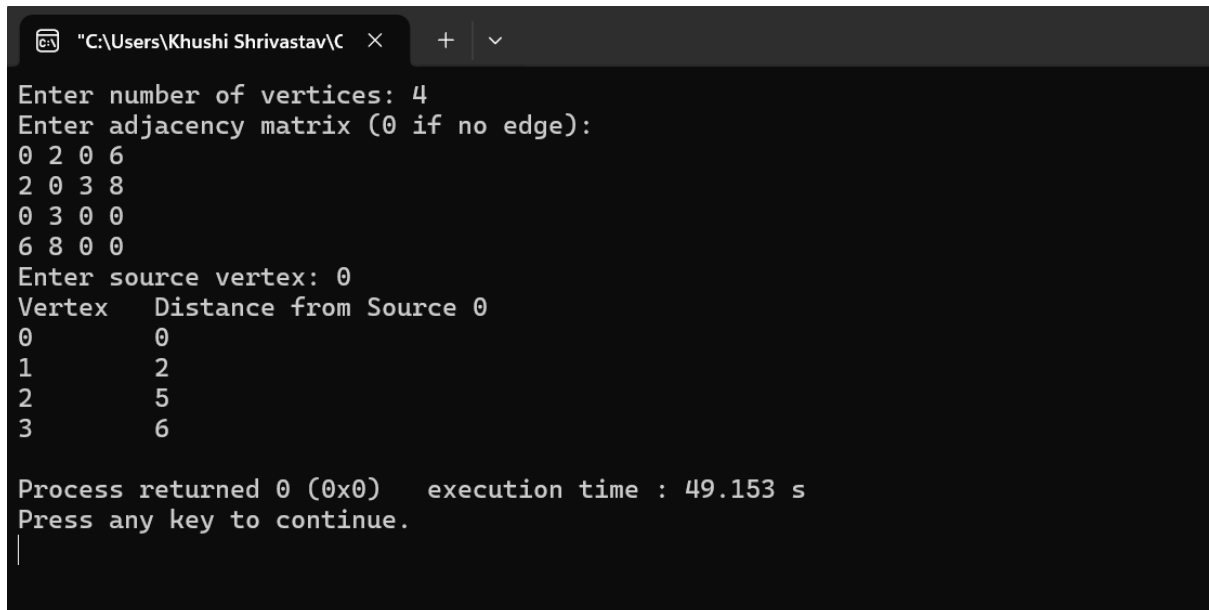
    int src;
    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, src, vertices);

    return 0;
}

```

Screenshot of the output:



A screenshot of a terminal window showing the execution of a C++ program. The window title is "C:\Users\Khushi Shrivastav\C". The program prompts the user to enter the number of vertices (4) and the adjacency matrix (0 if no edge). The matrix is: 0 2 0 6, 2 0 3 8, 0 3 0 0, 6 8 0 0. The source vertex is 0. The output shows the distance from source 0 for each vertex: 0, 2, 5, 6. The process returned 0 (0x0) and the execution time was 49.153 s. The prompt "Press any key to continue." is shown with a cursor.

```
"C:\Users\Khushi Shrivastav\C" × + v
Enter number of vertices: 4
Enter adjacency matrix (0 if no edge):
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Enter source vertex: 0
Vertex    Distance from Source 0
0         0
1         2
2         5
3         6

Process returned 0 (0x0)   execution time : 49.153 s
Press any key to continue.
|
```


Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

#include <stdbool.h>

#define MAX 20

int N;

int board[MAX][MAX];

bool isSafe(int row,int col){
    for(int i=0;i<row;i++){
        if(board[i][col])return false;
        if(col-(row-i)>=0&&board[i][col-(row-i)])return false;
        if(col+(row-i)<N&&board[i][col+(row-i)])return false;
    }
    return true;
}

void printBoard(){
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++)printf("%c ",board[i][j]?'Q':'.');
        printf("\n");
    }
    printf("\n");
}

void solve(int row){
    if(row==N){
        printBoard();
        return;
    }
    for(int col=0;col<N;col++){
        if(isSafe(row,col)){
```

```

        board[row][col]=1;
        solve(row+1);
        board[row][col]=0;
    }
}
}

int main(){
    printf("Enter the size of the board (N): ");
    scanf("%d",&N);
    if(N>MAX||N<1){
        printf("Invalid board size. Please enter between 1 and %d.\n",MAX);
        return 1;
    }
    solve(0);
    return 0;
}

```

Screenshot of the output:

```

C:\Users\Khushi Shrivastav\C >
Enter the size of the board (N): 4
. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

Process returned 0 (0x0)   execution time : 1.883 s
Press any key to continue.
|

```